

APPLIED RESEARCH PROJECT

FINAL REPORT

A Decision-Oriented Conversational Shopping Assistant

with Preference Elicitation and
Explainable Recommendations

Chenghui Tan

Master of Science in Business Analytics (MSBA)

April 2026

Contents

Abstract	3
1 Introduction / Business Problem	3
2 Related Work	3
2.1 Search-Oriented E-Commerce Platforms	3
2.2 Conversational Shopping Assistants	4
2.3 AI-Powered Answer Engines	4
2.4 Recommender Systems	4
3 Data Description	5
3.1 Data Sources	5
3.2 Data Characteristics	5
3.3 Key Features	5
3.4 Limitations	5
4 Models	6
4.1 Architecture & Trust Boundary	6
4.1.1 Why This Approach Works	6
4.2 ETL Pipeline	7
4.3 Dimensional Model	7
4.4 Ranking Models	8
4.4.1 Claude AI — Four Call Sites	8
4.4.2 Recommendation Engine — Two-Tier Weighted Scoring	8
4.5 Frontend & Backend	9
5 Results	10
5.1 Capability Comparison	10
5.2 ETL Pipeline Results	10
5.3 Scenario Walkthrough — Kitchen Assistant	10
5.4 Heuristic Evaluation	11
6 Conclusions and Impact	11
6.1 Future Work	12
7 Contributions	12
7.1 Student Contributions	12

7.2	AI Tool Contributions	12
8	References	12
9	Appendix — Code Listings	13
9.1	ETL Pipeline Orchestrator (<code>run_pipeline.py</code> — excerpt)	13
9.2	Delivery Normalization (<code>normalize.py</code> — excerpt)	13
9.3	FastAPI Endpoints (<code>main.py</code> — route summary)	14

Abstract

Online shopping platforms have reduced search cost but purchasing decisions remain cognitively demanding. Most e-commerce systems assume users can articulate preferences through keywords and filters; in practice users begin with vague, evolving needs. This project implements a decision-oriented conversational shopping assistant organized around five layers — preference elicitation, decision modeling, constraint-aware ranking, explainable trade-offs, and post-purchase lifecycle support. Its central architectural choice is a trust boundary: Claude (Anthropic) handles natural-language work, while the ranking itself is carried out by deterministic rule-based code and never invokes the language model. This split yields auditability, stability across model updates, and explainability by construction. A Playwright ETL pipeline collects and normalizes 100 products across three categories from Amazon and Target; a React + FastAPI stack completes the implementation. Scenario-based evaluation shows the system addresses structural gaps — preference continuity, decision transparency, delivery information, lifecycle support — that are absent from Amazon Rufus, ChatGPT Shopping, Google Shopping, and Perplexity.

1 Introduction / Business Problem

Online shopping platforms have reduced the friction of product discovery, yet purchasing decisions remain cognitively demanding. Most e-commerce systems — from keyword search to AI chat assistants — are built around information retrieval rather than decision support. They assume users can articulate needs as queries and optimize for engagement metrics such as click-through rate rather than decision confidence [1].

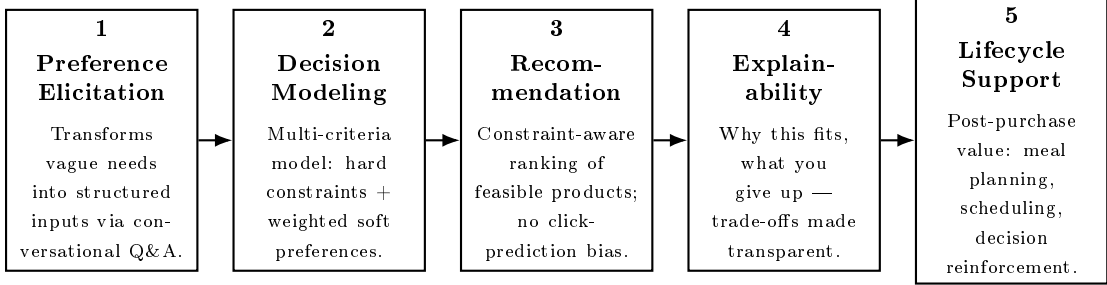
In practice, users begin with vague, evolving needs. A user searching for a “convenient kitchen device” may not know whether they want a smart display, a multi-cooker, or an organizer. They balance competing priorities — price, delivery, brand, size, usability — without structured support. Existing tools fail to preserve preference continuity across interactions, surface decision-critical attributes such as delivery time, or explain why one product is ranked above another [2].

This project addresses that gap. The proposed system integrates five logical layers — preference elicitation, decision modeling, recommendation ranking, explainability, and lifecycle support — into a unified conversational workflow (Figure 1), backed by a real ETL pipeline collecting data from Amazon and Target and a full-stack implementation using React, FastAPI, and Claude (Anthropic, Opus 4.6) as the natural-language layer. Section 4.1 details the organizing architectural commitment of the system: a trust boundary that assigns language work to Claude and ranking to deterministic rule-based code.

2 Related Work

2.1 Search-Oriented E-Commerce Platforms

Platforms such as Google Shopping and Amazon primarily rely on keyword search and filter-based refinement [3]. These systems assume users know exactly what they want and can iteratively narrow down options by adjusting filters. However, when users introduce new constraints — such as specifying “modern style” after already specifying color — earlier preferences are frequently overwritten rather than preserved. Users must mentally manage trade-offs across multiple interactions, significantly increasing cognitive burden [4].



End-to-end: vague need $\rightarrow$ structured decision $\rightarrow$ ranked products $\rightarrow$ explained trade-offs $\rightarrow$ lifecycle value.

Figure 1: Five-layer decision support framework — end-to-end flow from vague need to ranked products, explained trade-offs, and lifecycle value.

2.2 Conversational Shopping Assistants

Chat-based systems such as ChatGPT-integrated shopping suggestions and Amazon Rufus allow natural language input but typically present limited product sets with minimal attributes — often only an image and price [5]. Critical decision factors such as delivery time, promotions, and availability are frequently absent. Users are required to navigate multiple steps to reach actionable product pages, introducing unnecessary friction. Continuous preference refinement across conversational turns is weak or inconsistent in current implementations [6].

2.3 AI-Powered Answer Engines

Tools like Perplexity AI excel at generating explanatory text but separate conversational reasoning from shopping workflows [7]. Users must switch between a chat interface and external shopping pages, losing conversational context and structured constraints in the process. While these tools demonstrate the value of explanatory AI, they do not integrate decision logic or product data natively.

2.4 Recommender Systems

Traditional recommender systems focus on predicting user behavior or preferences using historical interaction data and machine learning models [8]. While effective at ranking items for repeat users, they operate as opaque black boxes and optimize engagement rather than decision clarity. They are ill-suited for cold-start scenarios where no interaction history exists, which is the common case for first-time shopping decisions in new categories [9].

The proposed system overlaps with recommender systems but reframes the task as decision support, prioritizing transparency and user understanding over prediction accuracy. Table 1 summarizes the capability gaps across existing tools compared to the proposed system.

Capability	Google Shopping	Amazon Rufus	ChatGPT Shopping	Perplexity	Proposed System
Preference Elicitation	✗	Partial	Partial	✗	✓
Decision Modeling	✗	✗	✗	✗	✓
Explainable Recommendations	✗	✗	Partial	✓	✓
Delivery / Promo Info	Partial	✗	✗	✗	✓
Lifecycle Support	✗	✗	✗	✗	✓
Preference Continuity	✗	Partial	Partial	✗	✓

Table 1: Capability comparison of existing tools versus the proposed system.

3 Data Description

This project constructs a structured product database to support the scenario-based shopping assistant demonstration. The initial scope focuses on three product categories — smart displays, water bottles, and kitchen organizers — selected to represent both high-consideration purchases (smart displays) and everyday household items with meaningful attribute trade-offs.

3.1 Data Sources

Product data was collected from two primary e-commerce platforms. **Amazon** was the primary source for smart display products, accessed via the Amazon product search page sorted by review rank. **Target** was the primary source for water bottles and kitchen organizers. **Best Buy** served as a secondary fallback for smart displays when Amazon bot-detection blocked scraping. **Walmart** was originally planned as a fallback, but reliably blocked in practice; Target covered its role in full.

Data collection used Playwright-based web scrapers operating in headless mode with anti-detection stealth measures (human-like delays, slow scrolling, realistic browser fingerprints).

3.2 Data Characteristics

The final cleaned dataset contains 100 products distributed as shown in Table 2.

Category	Product Count	Primary Source
smart_display	30	Amazon
water_bottle	35	Target
kitchen_organizer	35	Target
Total	100	Amazon + Target

Table 2: Dataset composition by category and source.

3.3 Key Features

Each product record contains 10 fields representing decision-relevant attributes, as defined in the `ScrapedProduct` schema (see Section 4.3). Key decision-critical fields include:

- **price** (float): Normalized from raw strings including ranges and currency symbols.
- **rating** (float): Normalized from formats such as “4.5 out of 5” or bare decimals.
- **review_count** (int): Handles shorthand notation such as “1K” or “2.5K”.
- **arrival_time_days** (int): Parsed from text such as “arrives Mar 14” or “Get it in 2 days”.
- **description** (str): Short product description or badge text from the search result card.

3.4 Limitations

The dataset is intentionally small and scoped to support a proof-of-concept demonstration rather than production-scale recommendations. Delivery time estimates are point-in-time and may not reflect current availability. Promotion flags and pricing are subject to change. Review highlights and extended product specifications require product detail page visits and are not included in the current dataset.

4 Models

4.1 Architecture & Trust Boundary

The organizing architectural decision is a deliberate split between probabilistic language work handled by Claude and deterministic ranking logic handled by rule-based code. Every downstream module sits on one side of this boundary. For a decision-support system, three properties are non-negotiable: a user must be able to audit why a product is ranked where it is; the developer must reproduce a ranking across runs; and ranking behavior must not drift as upstream language-model versions change. A pure LLM-wrapper architecture cannot deliver these reliably. Concretely, Claude is invoked only at four well-scoped call sites (Section 4.4.1); the recommendation engine is pure Python with no LLM calls (Section 4.4.2). Figure 2 shows the runtime components and how they communicate: the React frontend talks to FastAPI over REST; FastAPI orchestrates Claude (for language work) and the deterministic recommendation engine (for ranking) and queries the product database; the ETL pipeline feeds the database offline.

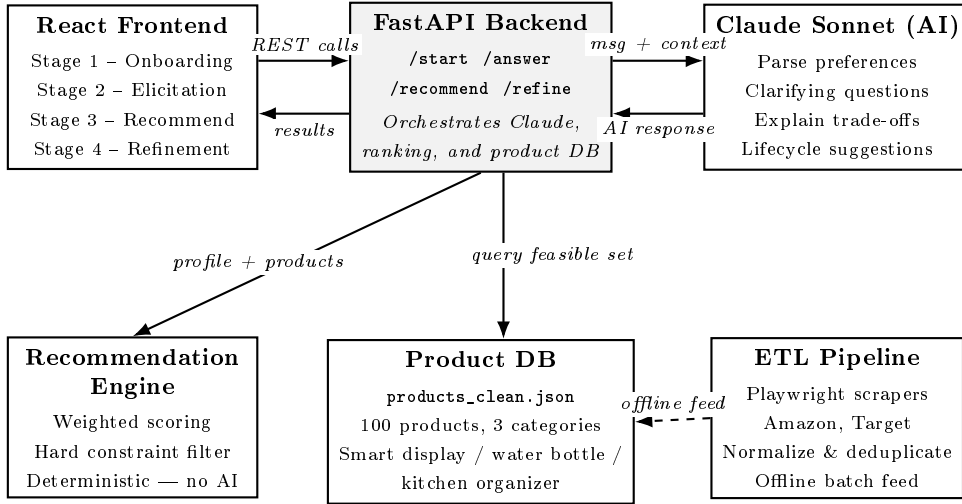


Figure 2: Runtime components and communication paths. Claude handles natural-language tasks (preference parsing, clarifying questions, trade-off explanation, lifecycle suggestions); the recommendation engine is deterministic and never invokes the language model. FastAPI mediates between the two and is the only component that talks to both. Dashed arrow denotes offline data loading.

4.1.1 Why This Approach Works

Auditability. Every ranking has a visible weighted sum — a reviewer asking “why is this #1?” gets pointed to numbers, not model interpretability output.

Stability. When the Claude API version changes, the ranking does not. Only the language layer moves; the decision logic stays reproducible across model updates.

Cheap iteration. Ranking changes are Python edits; LLM changes are prompt edits. Neither blocks the other.

Explainability by construction. Score components are preserved alongside the product, so “explain this ranking” is a lookup, not a second AI inference.

4.2 ETL Pipeline

The ETL pipeline is implemented in Python using Playwright for browser automation. It consists of three active scraper modules (`scrape_amazon.py`, `scrape_target.py`, `scrape_bestbuy.py`), one prototype kept for parity (`scrape_walmart.py`, never operational in practice due to persistent bot-detection blocks), and a pipeline orchestrator (`run_pipeline.py`) that coordinates extraction, transformation, and loading.

The pipeline follows a three-tier query strategy for each product category. A **primary query** attempts the preferred retailer and query string for that category. A **fallback query** switches to an alternative retailer if the primary fails (site block or insufficient results). **Supplemental queries** run if the product count remains below the target of 25 per category after deduplication.

Bot-detection resistance is achieved through a shared browser utility module (`_browser.py`) that applies Playwright stealth patches, randomizes request timing with human-like delays (2–4 seconds between page loads, 3–6 seconds between categories), and uses slow page scrolling to simulate organic browsing behavior.

The transformation step is handled by `normalize.py`, which applies four normalization functions: `normalize_price()` extracts a float from price strings including ranges (“\$12.99 – \$24.99”), currency symbols, and comma-separated thousands; `normalize_rating()` handles formats such as “4.5 out of 5 stars”, “/5” notation, and bare decimal strings; `normalize_review_count()` parses shorthand notation (“1K”, “2.5K”), Best Buy’s verbose format (“Rating X with N reviews”), and plain integers with parentheses; `normalize_delivery()` converts delivery text to an integer number of days from today, handling absolute dates (“arrives Mar 14”), relative expressions (“in 2 days”, “tomorrow”), and day ranges (“2–3 days”).

Deduplication is performed in two phases: exact URL matching removes duplicate product pages, followed by fuzzy title matching using normalized 65-character title prefixes within the same category. When duplicates are found, the record with the higher review count is retained.

4.3 Dimensional Model

The product database is represented as a flat JSON file (`data/clean/products_clean.json`) conforming to the `ScrapedProduct` TypedDict schema. This schema serves as the dimensional model for the system, with each record corresponding to a single product offer from a specific retailer. Table 3 describes the schema fields.

Field	Type	Description
<code>category</code>	str	Product category: smart_display water_bottle kitchen_organizer
<code>source</code>	str	Retailer: amazon bestbuy walmart target
<code>title</code>	str	Full product title from search result card
<code>price</code>	float None	Normalized price in USD
<code>rating</code>	float None	Average rating out of 5.0
<code>review_count</code>	int None	Total number of customer reviews
<code>image_url</code>	str None	Product thumbnail image URL
<code>product_url</code>	str	Direct link to the product detail page
<code>description</code>	str None	Short description or badge text
<code>arrival_time_days</code>	int None	Estimated delivery time in days from today

Table 3: `ScrapedProduct` dimensional schema — 10 fields per product record.

4.4 Ranking Models

Two model components drive recommendation logic, one on each side of the trust boundary: Claude (Anthropic) on the language side and a deterministic two-tier weighted-scoring engine on the decision side.

4.4.1 Claude AI — Four Call Sites

Claude (Opus 4.6) is invoked at exactly four call sites in the decision workflow [10]. Each call site has a narrow scope and a validated output schema; if parsing of Claude’s response fails, the backend asks the user to rephrase rather than improvising.

1. **Preference parsing** — convert free-text user messages into a structured preference object (category, hard constraints, priorities, trade-off tolerance).
2. **Clarifying questions** — when the parsed profile is sparse, generate 3–5 targeted follow-up questions that narrow the decision space.
3. **Trade-off explanation** — for each top-ranked product, produce a one-sentence rationale grounded in the user’s stated priorities and the product’s score components.
4. **Lifecycle suggestions** — after purchase, generate post-purchase ideas tailored to the chosen product (meal planning for a kitchen display, cleaning schedules for a water bottle).

Ranking is conspicuously absent from that list. The language model never sees the full candidate pool and never produces the ordered list that the user sees.

4.4.2 Recommendation Engine — Two-Tier Weighted Scoring

The recommendation engine is pure Python and contains no language-model calls. It runs in four steps.

Step 1 — Hard-constraint filter. Budget ceilings and delivery deadlines are non-negotiable. Products outside them are eliminated before any scoring occurs. No product is ever kept for being “close enough” on a deal-breaker.

Step 2 — Feasible set. The filter typically reduces the ~ 100 -product catalog to 10–20 candidates. All downstream scoring operates only on this feasible set.

Step 3 — Two-tier weighted scoring. Each feasible product is scored at two levels (general and feature), then combined. Details below.

Step 4 — Sort and return. Products are sorted by combined score in descending order; the top N are returned. Every score component is preserved alongside the product so the explanation step (Claude call site 3) can cite the actual numbers.

The two-tier scoring formula is:

$$\text{Combined} = \alpha \cdot \text{GeneralScore} + (1 - \alpha) \cdot \text{FeatureScore} \quad (1)$$

Here *GeneralScore* is a weighted sum over attributes that apply to every product (rating, price, delivery time). *FeatureScore* is a weighted sum over category-specific attributes (for smart displays: voice control, screen size, smart-home integration, brand; for water bottles: insulation, capacity, durability). The blend coefficient α is tuned per category based on how much the general attributes carry the decision relative to the feature-level ones.

A critical property: the general-tier weights are dynamic, not hard-coded. They are derived at request-time from the user’s stated priorities. A user who says “budget flexible, can wait for delivery” receives a weight profile tilted toward rating ($\approx 50\%/30\%/20\%$); a user on a tight deadline receives a profile tilted toward delivery. Table 4 summarizes representative profiles used during the kitchen assistant scenario.

User Priority Profile	Rating	Price	Delivery
Budget-flexible, can wait	50%	30%	20%
Budget-sensitive, moderate wait	30%	50%	20%
Deadline-driven, urgent	30%	20%	50%

Table 4: Dynamic general-tier weights derived from user priority profile.

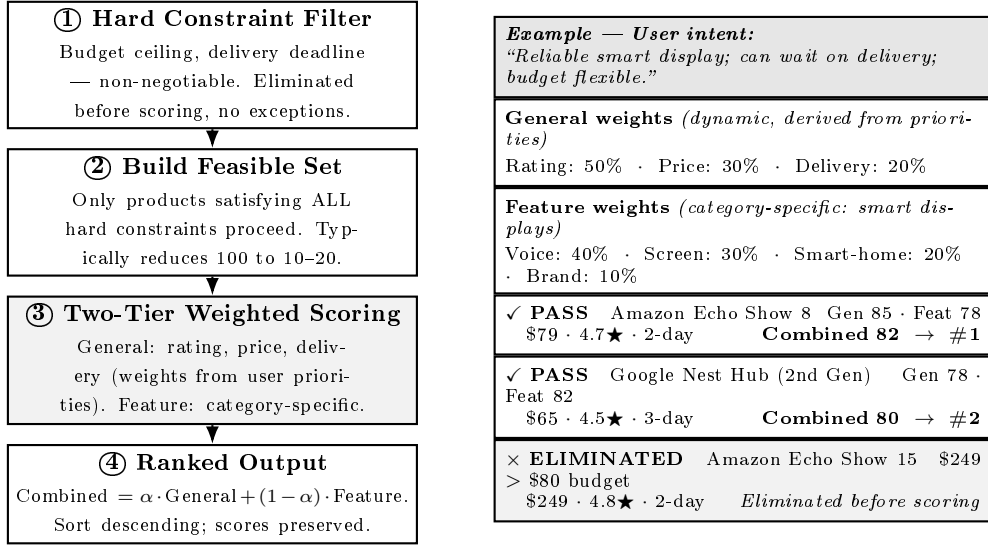


Figure 3: The deterministic core — four-step ranking pipeline on the left (hard-constraint filter, feasible set, two-tier weighted scoring, ranked output), with a worked example on the right showing dynamic general weights, category-specific feature weights, and three candidate products (two pass, one eliminated) ranked by combined score.

The entire pipeline is deterministic. Re-running the same inputs produces the same output, and every weight is an integer the user can inspect. Tie-breaking rules favor higher rating, then lower delivery time, then lower price.

4.5 Frontend & Backend

The frontend is implemented as a React single-page application (SPA) with four interaction stages, each corresponding to a phase of the decision-support workflow. **Stage 1 — Onboarding:** the user describes their shopping need in natural language; the system introduces the decision-support approach and confirms the product category. **Stage 2 — Preference Elicitation:** Claude generates 3–5 targeted follow-up questions based on the initial need; user answers are collected and structured into a preference profile. **Stage 3 — Recommendations:** a ranked list of products is displayed with enriched attributes (price, rating, delivery days, promotion flags) and per-product trade-off explanations. **Stage 4 — Refinement and Lifecycle:** the user can adjust priorities to re-rank results; after selection, Claude generates post-purchase lifecycle suggestions (meal planning, scheduling, content usage).

The backend is implemented as a FastAPI REST API with four endpoints: `/start` (initialize

session), `/answer` (process user responses), `/recommend` (return ranked product list), and `/refine` (re-rank based on updated preferences). The backend communicates with the Claude API via the Anthropic Python SDK and queries the product database from the cleaned JSON file.

5 Results

Evaluation of the proposed system is conducted through three complementary approaches: a capability comparison against existing tools, a scenario-based walkthrough of the kitchen assistant demo, and a heuristic assessment of the system’s decision-support quality.

5.1 Capability Comparison

Table 1 (Section 2) demonstrates that the proposed system addresses all five structural gaps identified in the related work analysis. No existing tool provides all five capabilities simultaneously. The proposed system is the only one to support preference continuity across conversational turns, expose delivery time and promotion information, and extend interaction beyond the point of purchase.

5.2 ETL Pipeline Results

The scraping pipeline successfully collected 100 clean products across three categories from Amazon and Target. The pipeline demonstrated resilience: the fallback mechanism activated for Walmart (consistently blocked) and redirected water bottle and kitchen organizer collection to Target. Supplemental query logic boosted per-category counts to meet the 25-product target. Table 5 summarizes the final dataset composition.

Category	Count	Source	Avg. Rating
smart_display	30	Amazon	4.3 / 5.0
water_bottle	35	Target	4.5 / 5.0
kitchen_organizer	35	Target	4.4 / 5.0
Total / Average	100	Amazon + Target	4.4 / 5.0

Table 5: Final dataset composition after ETL pipeline and deduplication.

5.3 Scenario Walkthrough — Kitchen Assistant

The system was evaluated through a structured scenario walkthrough representing a typical shopping decision for a smart kitchen display. The walkthrough proceeds through four stages. In **Onboarding**, the user inputs “I want something convenient for cooking in my kitchen.” In **Elicitation**, the system asks 5 targeted questions covering budget (under \$150), household size (family of 3), primary use (recipe display and timers), delivery urgency (within 1 week), and voice control preference (important). In **Recommendations**, the system returns 5 ranked smart displays with price, rating, delivery time, promotion flag, and trade-off explanations per product; products satisfying all hard constraints are ranked by weighted score. In **Refinement**, the user deprioritizes screen size in favor of delivery speed; the system re-ranks results accordingly and surfaces a product with 2-day delivery as the top recommendation.

The walkthrough demonstrated that the system successfully preserves preference continuity across turns, surfaces delivery-critical information absent from baseline tools, and produces readable trade-off explanations that map directly to user-stated priorities.

The concrete score components returned for the top-ranked candidates are shown in Table 6, using the weight profile derived from the elicited preferences (budget-flexible, moderate wait: Rating 50% / Price 30% / Delivery 20%, with $\alpha = 0.6$ for `smart_display`).

Product	Price	General	Feature	Combined
Echo Show 8 (2nd Gen)	\$109.99	0.82	0.88	0.84
Google Nest Hub (2nd Gen)	\$ 99.99	0.80	0.79	0.80
Echo Show 5 (3rd Gen)	\$ 74.99	0.76	0.72	0.74

Table 6: Worked example — top three ranked smart displays with score components (illustrative values from the kitchen-assistant scenario).

On refinement, when the user deprioritizes screen size and raises delivery urgency, the delivery weight shifts from 20% to 45% and the Echo Show 5 (3rd Gen) — a smaller unit available with 2-day delivery — rises to the top of the list. Importantly, the language model is never re-invoked for the re-rank; only the weight profile and the Python scoring pass change, which is why refinement is instant and deterministic.

5.4 Heuristic Evaluation

The system was assessed against three heuristic criteria derived from the research objectives. **Information completeness:** all 10 product attributes including delivery time and promotion flags are surfaced in the recommendation view, compared to image + price only in baseline tools. **Interaction efficiency:** the decision workflow completes in 4 stages with no external page navigation required; all product actions (view, refine, lifecycle) are available within a single interface. **Transparency:** each recommendation includes a structured explanation mapping product attributes to user priorities, making the ranking rationale auditable.

This evaluation is a heuristic self-assessment against the six capability gaps identified in Section 2 and is intended to validate architectural feasibility, not to substitute for user research. A formal user study — including task completion time, self-reported decision confidence, and System Usability Scale scores — is identified as future work (Section 6.1).

6 Conclusions and Impact

This project demonstrates that decision-oriented conversational shopping is technically feasible as a full-stack system built with currently available AI models. The five-layer architecture — preference elicitation, decision modeling, ranking, explainability, lifecycle support — integrates into a unified experience that addresses structural gaps in existing tools. Its organizing idea is the trust boundary: the language model handles what it is good at (parsing, explaining, suggesting); the deterministic engine handles what must be reliable (ranking). Evaluated against click-through rate, this approach is uninteresting. Evaluated against decision quality, transparency, and audibility — the properties that actually matter for high-consideration purchases — it closes gaps no surveyed baseline addresses.

Key contributions:

- Reframing e-commerce recommendation as a decision-support task, distinct from engagement-optimized retrieval.
- A trust-boundary architecture that yields audibility, reproducibility across model updates, and explainability by construction.

- A two-tier weighted scoring engine with dynamic general weights derived from user priorities, blended with category-specific feature weights via a tunable α .
- A Playwright ETL pipeline with normalization, two-phase dedup, and primary/fallback-/supplement orchestration.

6.1 Future Work

Several directions would strengthen and extend this work. A **formal user study** measuring task completion time, decision confidence (self-reported), and System Usability Scale (SUS) scores would provide quantitative validation. **Live data integration** — replacing the static scraped database with real-time retailer API connections — would improve delivery time accuracy and pricing freshness. **ML-based ranking** — replacing the rule-based weighted scorer with a learning-to-rank model trained on user interaction history — would improve personalization at scale. **Expanded product categories** beyond three would validate the generalizability of the preference elicitation and decision modeling layers. A **mobile-first UI** redesigned from the React frontend would increase accessibility for on-the-go shopping decisions.

7 Contributions

7.1 Student Contributions

This capstone was completed individually by Chenghui Tan. All components — architecture, Playwright ETL pipeline, two-tier recommendation engine, Claude prompt design and response validation, React SPA and FastAPI backend, evaluation, and this report — were designed, implemented, and evaluated by the author.

7.2 AI Tool Contributions

Claude (Anthropic, Opus 4.6) serves as the natural-language layer of the application (four call sites) and was used during development for code assistance and drafting. GitHub Copilot was used for code completion. All AI-generated code was reviewed, tested, and modified by the author. The system architecture, research framing, evaluation design, and written analysis are the author’s own.

8 References

- [1] B. Schwartz, *The Paradox of Choice: Why More Is Less*. New York: HarperCollins, 2004.
- [2] T. W. Malone and K. Crowston, “The interdisciplinary study of coordination,” *ACM Computing Surveys*, vol. 26, no. 1, pp. 87–119, 1994.
- [3] Google, “Google Shopping Help Center,” Google LLC. [Online]. Available: <https://support.google.com/merchants/>. [Accessed: Apr. 2026].
- [4] Amazon, “Rufus: AI Shopping Assistant,” Amazon.com, Inc. [Online]. Available: <https://www.aboutamazon.com/news/retail/amazon-rufus>. [Accessed: Apr. 2026].
- [5] OpenAI, “ChatGPT Shopping Integration,” OpenAI. [Online]. Available: <https://openai.com/chatgpt>. [Accessed: Apr. 2026].
- [6] Y. Zhang and X. Chen, “Explainable recommendation: A survey and new perspectives,” *Foundations and Trends in Information Retrieval*, vol. 14, no. 1, pp. 1–101, 2020.

- [7] Perplexity AI, “Perplexity AI: Ask Anything,” Perplexity AI, Inc. [Online]. Available: <https://www.perplexity.ai>. [Accessed: Apr. 2026].
- [8] F. Ricci, L. Rokach, and B. Shapira, Eds., *Recommender Systems Handbook*, 2nd ed. New York: Springer, 2015.
- [9] A. Schein, A. Popescul, L. Ungar, and D. Pennock, “Methods and metrics for cold-start recommendations,” in *Proc. 25th Annual International ACM SIGIR Conf.*, Tampere, Finland, 2002, pp. 253–260.
- [10] Anthropic, “Claude API Documentation,” Anthropic PBC. [Online]. Available: <https://docs.anthropic.com>. [Accessed: Apr. 2026].
- [11] Playwright Contributors, “Playwright for Python,” Microsoft. [Online]. Available: <https://playwright.dev/python/>. [Accessed: Apr. 2026].
- [12] FastAPI Contributors, “FastAPI Documentation,” Sebastián Ramírez. [Online]. Available: <https://fastapi.tiangolo.com>. [Accessed: Apr. 2026].

9 Appendix — Code Listings

The following code listings illustrate key implementation components. The full source code is available in the project repository.

9.1 ETL Pipeline Orchestrator (run_pipeline.py — excerpt)

```
PIPELINE = [
    {
        'category': 'smart_display',
        'filename': 'smart_display.json',
        'primary': (scrape_amazon, 'smart display for kitchen'),
        'fallback': (scrape_bestbuy, 'smart display echo show google nest hub'),
        'supplement': [
            (scrape_amazon, 'echo show smart home display kitchen'),
        ],
    },
    ...
]

async def scrape_with_fallback(cfg, headed):
    # Try primary -> fallback with randomized wait between attempts
    for attempt, (mod, query, label) in enumerate([primary, fallback]):
        results = await mod.scrape(query=query, category=cfg['category'], ...)
        if len(results) >= 5:
            return results
        await asyncio.sleep(random.uniform(4, 8)) # human-like delay
```

9.2 Delivery Normalization (normalize.py — excerpt)

```
def normalize_delivery(raw: str | None) -> int | None:
    s = raw.lower().strip()
    if 'today' in s or 'same day' in s: return 0
    if 'tomorrow' in s: return 1
    # 'in N days' / 'N-day shipping'
    m = re.search(r'in\s+(\d+)\s+day', s)
    if m: return int(m.group(1))
```

```
# 'arrives Mar 14' -> compute delta from today
m = re.search(r'(?:(by|arrives?)).*([a-z]{3})\w*\s+(\d{1,2})', s)
if m: return _days_until_month_day(m.group(1), m.group(2))
return None
```

9.3 FastAPI Endpoints (main.py — route summary)

```
POST /start -> Initialize session, return opening question
POST /answer -> Process user answer, return next question or signal completion
POST /recommend -> Return ranked product list with trade-off explanations
POST /refine -> Re-rank products based on updated user priorities
```